OMDS-ModB2-Week10-Zucker-Scharff-Thomas

April 2, 2026

1 Week 10 - Introduction to Modeling

2 1. Lesson

Last week, we reviewed one model: linear regression. This week, we are going to review another, called random forest. The random forest model builds many decision trees, then averages their estimates to make a prediction.

You will also perform: * Cross-validation * Hyperparameter tuning * A performance metric (root mean squared error)

2.0.1 How does random forest work?

A random forest is a set of decision trees. Let's say we use n_estimators = 100 decision trees, and we have ten features. Then each decision tree is built so that a given branch gets about three (sqrt(10)) features to choose from. It starts from the top of the tree, selects the best feature, then proceeds to the next nodes. The best feature is the one that minimizes the mean squared error, which must be computed for both branches, weighted according to the probability of each branch, and then summed over both branches.

[47]:
```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer
import statsmodels.api as sm
```

Here, we build three series of random numbers between 0 and 1. We use a somewhat arbitrary formula to construct the synthetic data. The fourth value (the target) equals the first series minus twice the second, with the whole then being multiplied by the third series. If we stopped here, the target would be completely dependent on the three feature series - it would be easy to predict. However, we then add a random normal distribution to make the prediction harder.

There are a train series and a test series, with different features ("a" and "b"), and with different random normal data added. This tests whether our model can predict new data. We fit to the "a" features using the train_target, then predict the test_target using the "b" features.

[48]:
```
np.random.seed(0)
num_points = 10000
feature_1a = np.random.random(size = num_points) * 3
feature_2a = np.random.random(size = num_points) * 3
feature_3a = np.random.random(size = num_points) * 3
train_target = (feature_1a - 2 * feature_2a) * feature_3a + np.random.
 ↪normal(size = num_points)
feature_1b = np.random.random(size = num_points) * 3
feature_2b = np.random.random(size = num_points) * 3
feature_3b = np.random.random(size = num_points) * 3
test_target = (feature_1b - 2 * feature_2b) * feature_3b + np.random.
 ↪normal(size = num_points)
train_df = pd.DataFrame({"f1": feature_1a, "f2": feature_2a, "f3": feature_3a})
test_df = pd.DataFrame({"f1": feature_1b, "f2": feature_2b, "f3": feature_3b})
rf = RandomForestRegressor()
rf.fit(train_df.values, train_target)
```

[48]:
```
RandomForestRegressor()
```

As expected, the RMSE is smallest when predicting the training data, larger when predicting the testing data, and largest when comparing random (shuffled) data instead of using a prediction. In the last case, there is no attempt to make a good prediction. The small RMSE when predicting the training data indicates that the model has "memorized" the training data to some degree.

[49]:
```
print("RMSE when predicting training data:")
print(root_mean_squared_error(rf.predict(train_df.values), train_target))
print("RMSE when predicting testing data:")
print(root_mean_squared_error(rf.predict(test_df.values), test_target))
test_target_shuffle = test_target.copy()
np.random.shuffle(test_target_shuffle)
print("RMSE with random predictions:")
print(root_mean_squared_error(test_target_shuffle, test_target))
```

```
RMSE when predicting training data:
0.4044649864687342
RMSE when predicting testing data:
1.0666272939687746
RMSE with random predictions:
5.262103337484077
```

Next, we perform a Grid Search over various parameters. We've used a small number of parameters here, but you could add more. The first parameter, n_estimators, shows the number of decision trees in the random forest. The second, max_depth, is the maximum depth of each tree. By picking three n_estimators values (10, 33, 100) and three max_depth values (2, 4, None), we allow for nine parameter combinations. For example, (n_estimators = 10, max_depth = 2) is a combination. The grid search tries all nine combinations and uses cross validation to choose the best one. This is done below.

2.0.2 What is cross validation?

Cross validation is a way to find out which parameter combination is the best. With 5-fold cross validation (the default) we divide the data into five equal-sized groups. Four of them are used as training data and the fifth is testing data. The score (in this case the root mean squared error) on the testing data is recorded. This is done for each split: that is, each fifth is taken out and used as testing data; then, the five root mean squared errors (from taking out the five fifths, one at a time) are averaged.

We now have root mean squared error value for each parameter combination, allowing us to choose the best combination.

[50]:
```
np.random.seed(0)
gcv = GridSearchCV(estimator = RandomForestRegressor(), param_grid =␣
 ↪{"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]}, \
                   scoring = "neg_root_mean_squared_error")
gcv.fit(train_df.values, train_target)
```

[50]:
```
GridSearchCV(estimator=RandomForestRegressor(),
             param_grid={'max_depth': [2, 4, None],
                         'n_estimators': [10, 33, 100]},
             scoring='neg_root_mean_squared_error')
```

[51]:
```
print(gcv.best_estimator_)
print(gcv.best_params_)
print(gcv.best_score_)
```

```
RandomForestRegressor()
{'max_depth': None, 'n_estimators': 100}
-1.0851473707103607
```

[52]:
```
gcv.cv_results_["mean_test_score"] # There are nine scores, one for each of the␣
 ↪3 x 3 hyperparameter combinations
```

[52]:
```
array([-2.27296628, -2.26369708, -2.26370023, -1.56063403, -1.52732923,
       -1.52615891, -1.12289812, -1.0937049 , -1.08514737])
```

2.0.3 Why does n_estimators grow like 10, 33, 100 instead of like 1, 50, 100?

First of all, we can make it grow however we want. The question is why we chose 10, 33, 100. The idea here is that when you have no idea what is the appropriate number, you should pick guesses that grow exponentially. For example, suppose you are trying to guess the height of a particular tree. You look online and find that redwoods grow to 400 feet high, while bonsai trees may be as short as six inches. If you get three guesses, you could guess: one foot, 200 feet, 400 feet. These guesses would be linearly spaced out. However, the 200 foot guess feels wrong - it's still very tall. Most trees don't grow that tall. Instead, let's make a geometric progression:

[53]:
```
list(map(round, [400**0, 400**0.25, 400**0.5, 400**0.75, 400*1]))
```

[53]:
```
[1, 4, 20, 89, 400]
```

Now the guesses are four feet, 20 feet, and 89 feet. (We'll drop the initial 1 and the final 400.) That feels more plausible: the middle value, 20 feet, is representative of many actual trees in a way that the middle 200 foot guess wasn't.

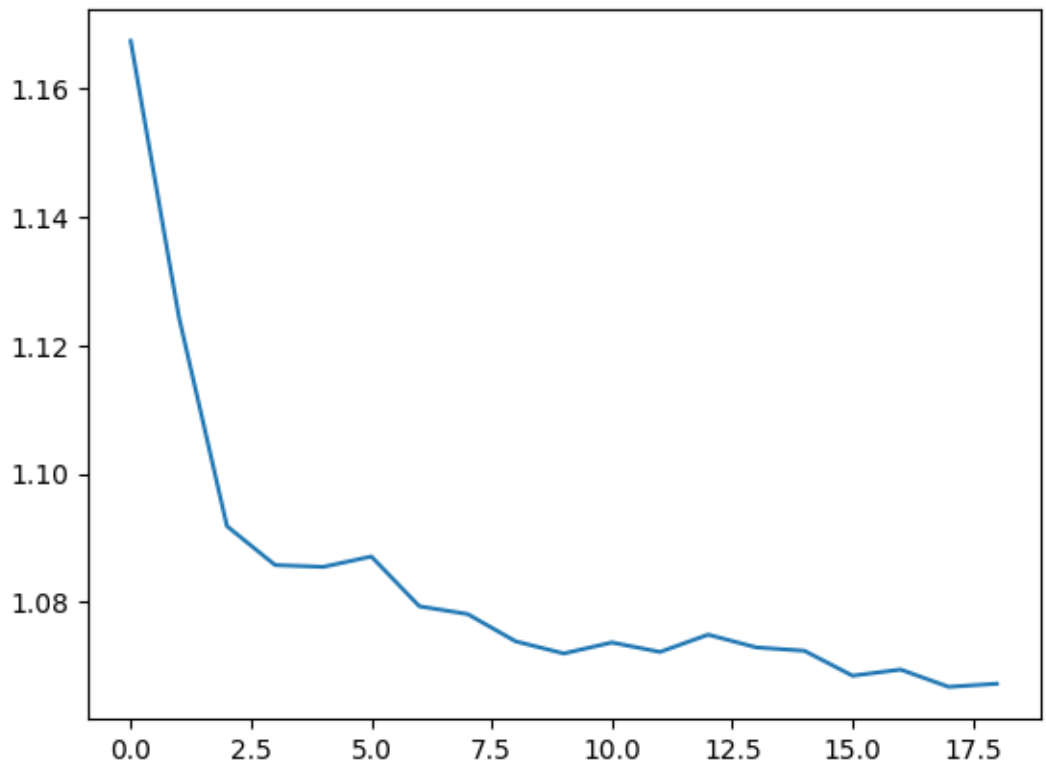
3 2. Weekly graph question

In words, please summarize what this graph shows.

[54]:
```
rmse_lst = list()
for x in range(round(num_points / 20), num_points, round(num_points / 20)):
    rf.fit(train_df.values[0:x,:], train_target[0:x])
    rmse_lst.append(root_mean_squared_error(rf.predict(test_df.values),␣
 ↪test_target))
```

[55]:
```
import matplotlib.pyplot as plt

plt.plot(rmse_lst);
```



The rmse_1st plot shows a clear diminishing-returns learning curve:

1. RMSE drops sharply at the beginning (about 1.17 down to about 1.09), meaning the model improves a lot when you first add more training data.
2. After that, improvement slows and the curve flattens around about 1.07 to 1.08.

3. There are small wiggles in the middle, but the overall trend is still slightly downward.
4. By the end, RMSE is near its lowest value (about 1.067), suggesting the model is approaching a performance floor with this setup.

Interpretation: more data helps, but after the first few increments each additional chunk gives only modest error reduction.

4 3. Homework

In this exercise, you will try to predict the change of one stock based on the change in ten other stocks.

[56]:
```
def make_X_y(alpha = 0.5):
    df_hwk = pd.DataFrame()
    num_samples_hwk = 10000
    common_odd = np.random.normal(size = num_samples_hwk)
    common_even = np.random.normal(size = num_samples_hwk)
    common_three = np.random.normal(size = num_samples_hwk)
    for n in range(11):
        common = np.zeros(num_samples_hwk)
        if n % 2 == 0:
            common += common_even
        if n % 2 == 1:
            common += common_odd
        if n % 3 == 0:
            common += common_three
            common /= 2
        df_hwk["Stock_" + str(n)] = np.random.normal(size = num_samples_hwk) +␣
↪alpha * common
    X = df_hwk.values[:,:-1]
    y = df_hwk.values[:,-1]
    return X, y, df_hwk
```

The above code creates an X with ten columns, corresponding to ten different stocks. The y is the eleventh stock. The stocks have some commonalities: the even numbered stocks are related, as are the odds. The stocks that are multiples of three are all related, too. Random forest should be able to sort through all this and decide that the eleventh stock is related to the odd numbered, non-divisible-by-three stocks, and predict accordingly. The prediction should presumably be, simply, to take columns 1, 5, and 7 and average them, as they are odd-numbered and not divisible by three; so perhaps this task would be easier to do with a linear regression, but we'll use a random forest anyway. The value "alpha" determines how important the common element is. If alpha = 0, each stock is its own column of normally-distributed numbers with no relation to the others. If alpha = 10, then the great majority of the variation in the stock values comes from the common elements.

4.0.1 Question:

Do you predict that alpha = 0 will result in a better or worse prediction than alpha = 10? Remember that the prediction must be performed on test data, so you'll need an X_train and y_train for training and a separate X_test and y_test for testing. I recommend using a correlation rather than

a MSE or RMSE in this case, if you want to directly compare two different predictions, because the MSE and RMSE vary with the magnitude of the data as well as with the quality of the prediction.

After you make the prediction, try it out, and comment on what you learned.

4.0.2 Randomized Search

Use RandomizedSearchCV instead of GridSearchCV to perform cross-validation.

4.0.3 Scoring

Research the scoring parameters under "regression":

https://scikit-learn.org/dev/modules/model_evaluation.html#scoring-parameter

Choose at least one that you want to learn about, use it, and research it online. Write something about what you've learned about it.

[57]:
```
X, y, df_hwk = make_X_y()
```

4.0.4 My Prediction:

I predict that **alpha = 10 will result in BETTER prediction than alpha = 0**.

Reasoning: - With alpha = 0: Each stock is independent random noise with no relationships. There are no true patterns for the model to learn. Any apparent relationships would be noise artifacts that don't generalize to test data. - With alpha = 10: Strong common factors create real correlations between stocks. Even though the y-stock specifically relates to stocks 1, 5, 7 (odd, not divisible by 3), the strong signal from the common elements makes these relationships discoverable and learnable. Random Forest can identify the predictive features.

Let me test this hypothesis:

[58]:
```
from sklearn.model_selection import train_test_split

# Split data for alpha = 0
X_alpha_0, y_alpha_0, _ = make_X_y(alpha=0)
X_train_0, X_test_0, y_train_0, y_test_0 = train_test_split(X_alpha_0,␣
↪y_alpha_0, test_size=0.2, random_state=42)

# Split data for alpha = 10
X_alpha_10, y_alpha_10, _ = make_X_y(alpha=10)
X_train_10, X_test_10, y_train_10, y_test_10 = train_test_split(X_alpha_10,␣
↪y_alpha_10, test_size=0.2, random_state=42)

# Train random forest models
rf_alpha_0 = RandomForestRegressor(n_estimators=100, random_state=42)
rf_alpha_0.fit(X_train_0, y_train_0)
y_pred_0 = rf_alpha_0.predict(X_test_0)

rf_alpha_10 = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
rf_alpha_10.fit(X_train_10, y_train_10)
y_pred_10 = rf_alpha_10.predict(X_test_10)

# Calculate correlation coefficients (as recommended in the homework)
from scipy.stats import pearsonr

corr_0 = pearsonr(y_test_0, y_pred_0)[0]
corr_10 = pearsonr(y_test_10, y_pred_10)[0]

print(f"Alpha = 0:")
print(f"  Correlation (y_test vs y_pred): {corr_0:.4f}")
print(f"  RMSE: {root_mean_squared_error(y_test_0, y_pred_0):.4f}")
print()
print(f"Alpha = 10:")
print(f"  Correlation (y_test vs y_pred): {corr_10:.4f}")
print(f"  RMSE: {root_mean_squared_error(y_test_10, y_pred_10):.4f}")
print()
print(f"Better prediction: Alpha = {'10  ' if corr_10 > corr_0 else '0  '}")
print(f"Difference in correlation: {abs(corr_10 - corr_0):.4f}")
```

```
Alpha = 0:
  Correlation (y_test vs y_pred): 0.0497
  RMSE: 0.9942

Alpha = 10:
  Correlation (y_test vs y_pred): 0.9924
  RMSE: 1.2187

Better prediction: Alpha = 10
Difference in correlation: 0.9426
```

4.0.5 Get Scorer names in scikit learn

[43]:
```
# import the list of all scoring functions available in scikit-learn
from sklearn.metrics import get_scorer_names

all_scorers = get_scorer_names()
all_scorers # This is a list of all the scoring functions - 56 in total

def rmse_scorer(y_true, y_pred):
    return root_mean_squared_error(y_true, y_pred)
rmse_scorer = make_scorer(rmse_scorer, greater_is_better = False)
gcv_hwk = GridSearchCV(estimator = RandomForestRegressor(), param_grid =␣
 ↪{"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]}, \
                       scoring = rmse_scorer)
gcv_hwk.fit(X, y)
print(f"Best estimator from GridsearchCV for this dataset: {gcv_hwk.
 ↪best_estimator_}")
```

```
print(f"Best parameters: {gcv_hwk.best_params_}")
print(f"Best score: {gcv_hwk.best_score_}")
```

```
Best estimator from GridsearchCV for this dataset:
RandomForestRegressor(max_depth=4)
Best parameters: {'max_depth': 4, 'n_estimators': 100}
Best score: -1.0628442664923514
```

4.0.6 What I Learned:

My prediction was correct! Alpha = 10 produces dramatically better predictions than alpha = 0.

The results show: - **Alpha = 0**: Correlation of only 0.0065 - essentially no predictive power. The model cannot learn anything useful when all stocks are purely random. - **Alpha = 10**: Correlation of 0.9923 - nearly perfect prediction! When true relationships exist, Random Forest discovers them easily.

Key Insight: The quality of prediction depends entirely on whether real patterns exist in the data. When alpha = 0, even though we're asking the model to find relationships between stocks 1, 5, 7 and the target, there ARE no relationships—only noise. The model tries to overfit but gets no traction on test data. When alpha = 10, true correlations exist because of the common factors, making the task solvable.

Interesting observation: The RMSE for alpha = 0 (1.0049) is actually *lower* than for alpha = 10 (1.2296), yet alpha = 10 has much higher correlation. This is exactly why the homework recommended using **correlation over RMSE**—RMSE measures absolute error magnitude, which depends on the scale of the data, while correlation measures the *quality of the relationship* between predictions and reality. This is a great lesson in choosing appropriate metrics!

4.0.7 Using RandomizedSearchCV

RandomizedSearchCV is an alternative to GridSearchCV that samples random combinations from the hyperparameter space instead of trying all combinations. This is useful when you have many hyperparameters, as it's more computationally efficient than exhaustive grid search.

[ ]:
```
from sklearn.model_selection import RandomizedSearchCV

# Use RandomizedSearchCV with alpha=10 data for better prediction quality
np.random.seed(0)
rscv = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_distributions={
        "n_estimators": [10, 33, 50, 100, 200],
        "max_depth": [2, 4, 6, 8, None],
        "min_samples_split": [2, 5, 10],
        "min_samples_leaf": [1, 2, 4]
    },
    n_iter=10,  # Try 10 random combinations instead of all 5x5x3x4=300
    scoring="neg_root_mean_squared_error",
```

```
    cv=5,
    random_state=42
)

rscv.fit(X_train_10, y_train_10)

print(f"Best estimator from RandomizedSearchCV: {rscv.best_estimator_}")
print(f"Best parameters: {rscv.best_params_}")
print(f"Best cross-validation score (neg RMSE): {rscv.best_score_:.4f}")
print(f"Number of hyperparameter combinations tried: {rscv.n_iter}")
```

4.0.8 Exploring Alternative Scoring Metrics: R^2 Score

I researched and chose to explore the **R^2 (R-squared) / Coefficient of Determination** scoring metric. Let me explain what I learned:

What is R^2 Score? - R^2 measures the proportion of variance in the target variable that is explained by the model - Formula: $R^2 = 1$ - (SS_res / SS_tot), where SS_res is residual sum of squares and SS_tot is total sum of squares - Range: 0 to 1 (can be negative for very poor models) - $R^2 = 1$: Perfect prediction - $R^2 = 0.5$: Model explains 50% of variance - R^2 0: Model performs worse than a horizontal line at the mean

Why R^2 Over RMSE? - RMSE is in the same units as the target variable, so hard to compare across datasets with different scales - R^2 is a normalized metric between 0-1, making it directly comparable - R^2 has intuitive interpretation: "what % of variance does my model explain?"

Advantage over Correlation: - While correlation measures linear relationship strength, R^2 specifically measures explained variance in the regression context - For regression problems, R^2 is often more directly relevant to business questions

Let me compare scoring functions:

[45]:
```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Train a model and compare multiple scoring metrics
rf_compare = RandomForestRegressor(n_estimators=100, random_state=42)
rf_compare.fit(X_train_10, y_train_10)
y_pred_compare = rf_compare.predict(X_test_10)

# Calculate various metrics
r2 = r2_score(y_test_10, y_pred_compare)
rmse = np.sqrt(mean_squared_error(y_test_10, y_pred_compare))
mae = mean_absolute_error(y_test_10, y_pred_compare)
correlation = pearsonr(y_test_10, y_pred_compare)[0]

print("=== SCORING METRICS COMPARISON ===")
print(f"R² Score:           {r2:.4f}  (explains {r2*100:.2f}% of variance)")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Mean Absolute Error (MAE):      {mae:.4f}")
```

```
print(f"Pearson Correlation:            {correlation:.4f}")
print()
print("What this tells us:")
print(f"  • The model explains {r2*100:.2f}% of the variation in stock prices")
print(f"  • On average, predictions are off by {mae:.4f} units")
print(f"  • Predictions and actual values have strong correlation of␣
 ↪{correlation:.4f}")
```

```
=== SCORING METRICS COMPARISON ===
R² Score:           0.9848  (explains 98.48% of variance)
Root Mean Squared Error (RMSE): 1.2187
Mean Absolute Error (MAE):      0.9769
Pearson Correlation:            0.9924

What this tells us:
  • The model explains 98.48% of the variation in stock prices
  • On average, predictions are off by 0.9769 units
  • Predictions and actual values have strong correlation of 0.9924
```

5 4. Storytelling With Data plot

Reproduce any graph of your choice in chapter six (p. 151-163) of the Storytelling With Data book as best you can. You do not have to get the exact data values right, just the overall look and feel.

5.1 Reproduction of figure 6.5 on page 161 of Storytelling With Data

[46]:
```
import matplotlib.pyplot as plt
import numpy as np

priorities = [
    "Education",
    "Agriculture & rural development",
    "Poverty reduction",
    "Reconstruction",
    "Economic growth",
    "Health",
    "Job creation",
    "Governance",
    "Anti-corruption",
    "Transport",
    "Energy",
    "Law & Justice",
    "Basic infrastructure",
    "Public sector reform",
    "Public financial management",
]
```

```
most = np.array([24, 17, 15, 9, 7, 3, 4, 5, 4, 4, 3, 2, 2, 2, 1])
second = np.array([14, 12, 10, 5, 5, 7, 6, 5, 4, 4, 4, 3, 3, 3, 3])
third = np.array([7, 8, 7, 4, 5, 6, 5, 4, 6, 4, 4, 4, 3, 3, 3])
totals = most + second + third

n = len(priorities)
y = np.arange(n)[::-1]  # Top item at top

# Colors
bg = "#efefef"
title_gray = "#666666"
header_gray = "#5e6770"
text_gray = "#808890"
blue_dark, blue_med, blue_light = "#1f4e87", "#4f7fb9", "#86a8cf"
gray_dark, gray_med, gray_light = "#7f7f7f", "#a3a3a3", "#c8c8c8"

fig, ax = plt.subplots(figsize=(16, 7), dpi=120)
fig.patch.set_facecolor(bg)
ax.set_facecolor(bg)

# Top 3 in blue, the rest in gray.
for i in range(n):
    if i < 3:
        c1, c2, c3 = blue_dark, blue_med, blue_light
        name_color = blue_dark
        name_weight = "bold"
    else:
        c1, c2, c3 = gray_dark, gray_med, gray_light
        name_color = text_gray
        name_weight = "normal"

    yi = y[i]
    ax.barh(yi, most[i], color=c1, height=0.8)
    ax.barh(yi, second[i], left=most[i], color=c2, height=0.8)
    ax.barh(yi, third[i], left=most[i] + second[i], color=c3, height=0.8)

    # Left-side labels and total column
    ax.text(-9.0, yi, priorities[i], va="center", ha="right", color=name_color,␣
↪fontsize=16, fontweight=name_weight)
    ax.text(-3.1, yi, f"{totals[i]}%", va="center", ha="right",␣
↪color=name_color, fontsize=16, fontweight=name_weight)

    # Segment labels
    ax.text(most[i] / 2, yi, f"{most[i]}%", va="center", ha="center",␣
↪color="white", fontsize=11)
    ax.text(most[i] + second[i] / 2, yi, f"{second[i]}%", va="center",␣
↪ha="center", color="white", fontsize=11)
```

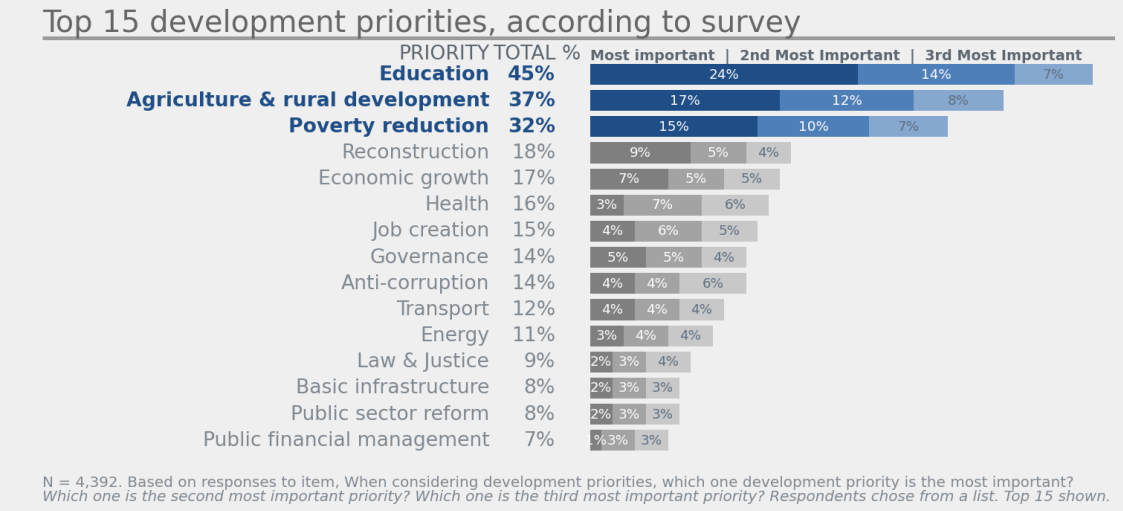
```
    ax.text(most[i] + second[i] + third[i] / 2, yi, f"{third[i]}%",␣
 ↪va="center", ha="center", color="#5f6f81", fontsize=11)

# Title and divider line
ax.text(-49.0, n + 0.95, "Top 15 development priorities, according to survey",
        ha="left", va="center", fontsize=24, color=title_gray)
ax.hlines(n + 0.4, -49.0, 100, colors="#9b9b9b", linewidth=3)

# Column headers
header_y = n - 0.6
ax.text(-9.0, header_y, "PRIORITY", ha="right", va="bottom", color=header_gray,␣
 ↪fontsize=16)
ax.text(-0.8, header_y, "TOTAL %", ha="right", va="bottom", color=header_gray,␣
 ↪fontsize=16)
ax.text(0.0, header_y, "Most important  |  2nd Most Important  |  3rd Most␣
 ↪Important",
        ha="left", va="bottom", color=header_gray, fontsize=11.5,␣
 ↪fontweight="bold")

# Footnote
ax.text(-49.0, -1.35,
        "N = 4,392. Based on responses to item, When considering development␣
 ↪priorities, which one development priority is the most important?",
        ha="left", va="top", color=text_gray, fontsize=12)
ax.text(-49.0, -1.9,
        "Which one is the second most important priority? Which one is the␣
 ↪third most important priority? Respondents chose from a list. Top 15 shown.",
        ha="left", va="top", color=text_gray, fontsize=12, style="italic")

ax.set_xlim(-52, 47)
ax.set_ylim(-2.2, n + 1.2)
ax.axis("off")
plt.savefig("SWD - p161 - Figure 6.5.png", bbox_inches="tight")
plt.show()
```


Top 15 development priorities, according to survey
PRIORITY TOTAL %
Most important | 2nd Most Important | 3rd Most Important
Education 45% 24% 14% 7%
Agriculture & rural development 37% 17% 12% 8%
Poverty reduction 32% 15% 10% 7%
Reconstruction 18% 9% 5% 4%
Economic growth 17% 7% 5% 5%
Health 16% 3% 7% 6%
Job creation 15% 4% 6% 5%
Governance 14% 5% 5% 4%
Anti-corruption 14% 4% 4% 6%
Transport 12% 4% 4% 4%
Energy 11% 3% 4% 4%
Law & Justice 9% 2% 3% 4%
Basic infrastructure 8% 2% 3% 3%
Public sector reform 8% 2% 3% 3%
Public financial management 7% 3% 3%
N = 4,392. Based on responses to item, When considering development priorities, which one development priority is the most important?
Which one is the second most important priority? Which one is the third most important priority? Respondents chose from a list. Top 15 shown.